PES UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Computer Networks Laboratory (UE23CS252) • Mini-Project Report

PROJECT #7 • TEAM 5

Packet Loss Detection and Recovery over Software-Defined Networks (Stop-and-Wait ARQ & Dynamic OpenFlow Path Failover)

Student Name	SRN
Arvind Namaji	PES1UG25CS102
Anirudh S	PES1UG25CS069
Bapathi Venkata Sri Abhivaran Reddy	PES1UG25CS122

1. ABSTRACT

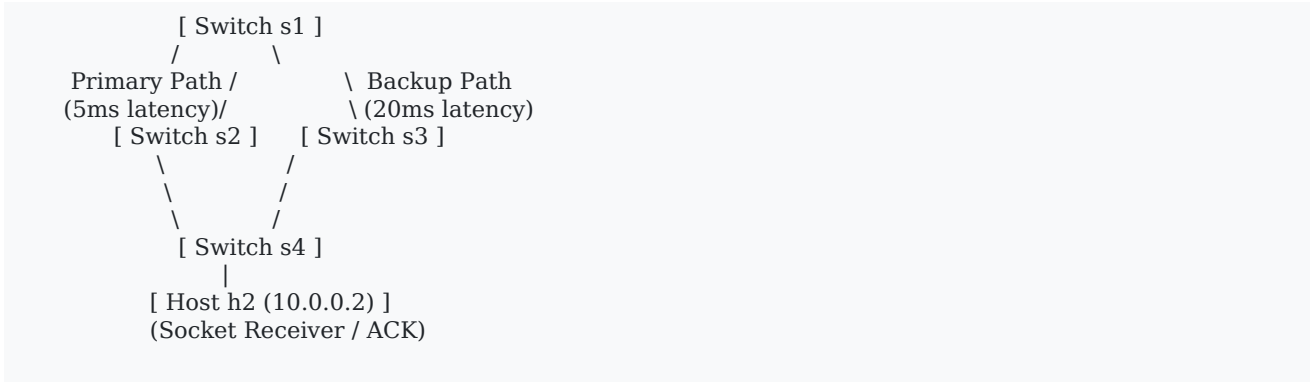
In traditional computer networks, the User Datagram Protocol (UDP) offers lightweight, connectionless datagram transport without delivery, ordering, or data integrity assurances. In contrast, standard Transmission Control Protocol (TCP) hides internal error detection and flow recovery within the operating system kernel. This project designs, implements, and benchmarks a two-tier reliable networked application that integrates custom transport-layer protocol engineering with Software-Defined Networking (SDN) in a virtualized Mininet environment.

At the transport layer, we engineered an alternating-bit Stop-and-Wait Automatic Repeat reQuest (ARQ) protocol over raw UDP sockets featuring a strict 12-byte binary header, CRC-32 integrity validation, and non-blocking timeout retransmissions. At the network layer, we configured a redundant multi-path mesh topology across four Open vSwitch (OVS) nodes operated under OpenFlow 1.3 rules. A custom SDN controller dynamically tracks physical port statuses, automatically rerouting traffic from a broken primary link (s1 <-> s2) to an alternate backup link (s1 <-> s3 <-> s4) in under 0.5 seconds, ensuring uninterrupted transport delivery without session termination.

2. SYSTEM ARCHITECTURE & PACKET SPECIFICATION

2.1 Multi-Path Mesh Topology in Mininet

The physical-layer emulation is implemented inside Mininet using Open vSwitch bridges configured in secure fail mode (failMode='secure'). Traffic flows between host h1 (Sender) and host h2 (Receiver) across two disjoint paths:



• Host h1 (10.0.0.1, Port 1 on s1): Transmits UDP packets as the Stop-and-Wait sender.
• Primary Path (s1 <-> s2 <-> s4): 10 Mbps bandwidth, 5 ms propagation delay (Mininet TCLink).
• Backup Path (s1 <-> s3 <-> s4): 10 Mbps bandwidth, 20 ms propagation delay for failover redundancy.
• Host h2 (10.0.0.2, Port 1 on s4): Listens on UDP port 12000, validates checksums, and sends ACKs.

2.2 Custom Binary Header Framing Specification

To avoid text parsing overhead, all datagrams are framed into binary format using Python's struct module adhering to Big-Endian Network Byte Order ('!IIHH'):

Field Name	Size	Data Type	Offset	Protocol Function
seq_num	4 Bytes	uint32	0 – 3	1-Bit alternating sequence flag (0 or 1)
ack_num	4 Bytes	uint32	4 – 7	Sequence number acknowledged by the receiver
flags	2 Bytes	uint16	8 – 9	Bitmask: 0x01 (DATA), 0x02 (ACK), 0x04 (FIN)
checksum	2 Bytes	uint16	10 – 11	Lower 16 bits of CRC-32 computed on payload
payload	Variable	Raw Bytes	12 – N	Application data chunk (e.g., 6 bytes/packet)

3. PROTOCOL LOGIC & DYNAMIC CONTROLLER DESIGN

3.1 Alternating-Bit Stop-and-Wait ARQ State Machine

Because Stop-and-Wait ARQ operates with a sliding window size W = 1, exactly one unacknowledged datagram exists in transit. This allows a 1-bit sequence number (0 <-> 1) to distinguish new packets from duplicate retransmissions with zero ambiguity:

• Sender Logic: Transmits chunk with active sequence bit (Seq 0). Starts a socket timeout timer (2.0s). Upon receiving ACK 0, it clears the timer, flips to Seq 1, and transmits the next chunk. If a timeout occurs, it triggers retransmission up to an 8-attempt ceiling.
• Receiver Logic: Computes CRC-32 on payload. If checksum is valid and seq_num matches expected_seq, it writes to the buffer, returns ACK, and toggles expected_seq. If seq_num is duplicate, it discards data to prevent buffer corruption and re-sends the previous ACK.

3.2 SDN OpenFlow Controller & Link Monitoring

The SDN controller (sdn_controller.py) communicates with Open vSwitch via OpenFlow 1.3. It inspects switch ports every 500 ms. When an administrator triggers a link drop, the controller detects PORT_DOWN on s1-eth2, purges old flow entries, and installs high-priority alternate routes through switch s3.

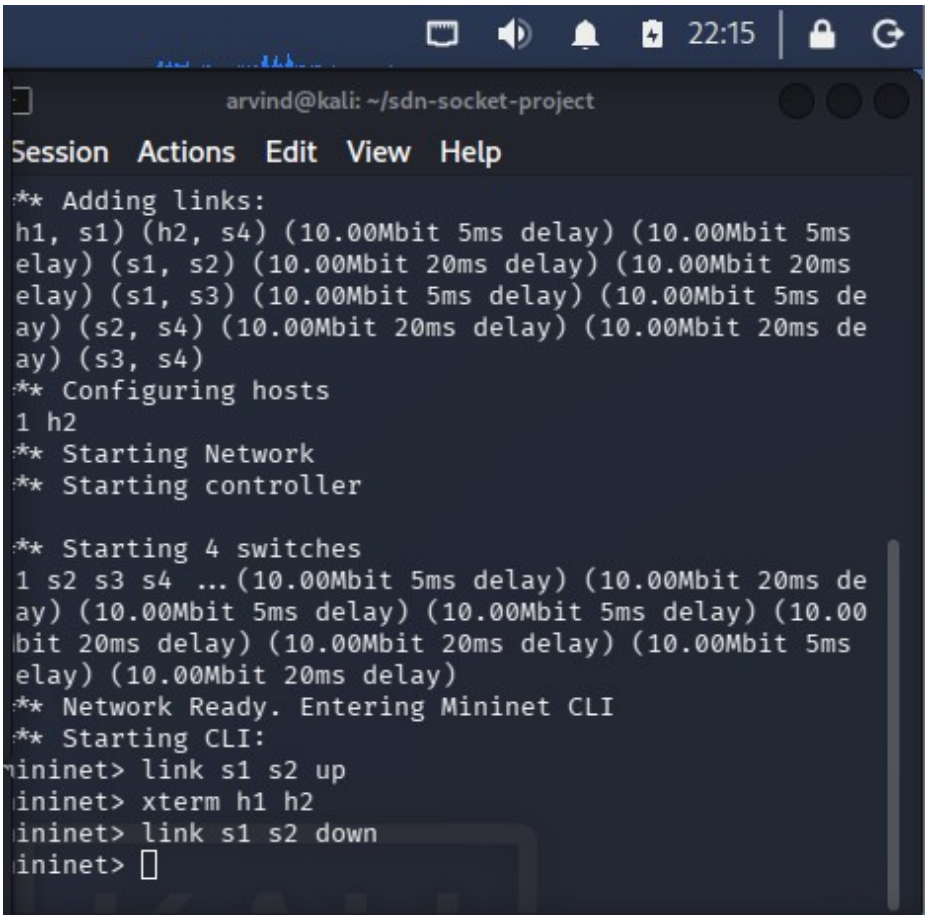


Figure 1: Mininet Topology Setup and Runtime Failure Injection ('link s1 s2 down')

4. EXPERIMENTAL VERIFICATION & ROBUSTNESS TESTING

During active socket streaming of a 15-chunk string from host h1 to h2, a physical link disruption was injected by bringing s1-eth2 down. The system successfully executed end-to-end failover recovery without application failure.

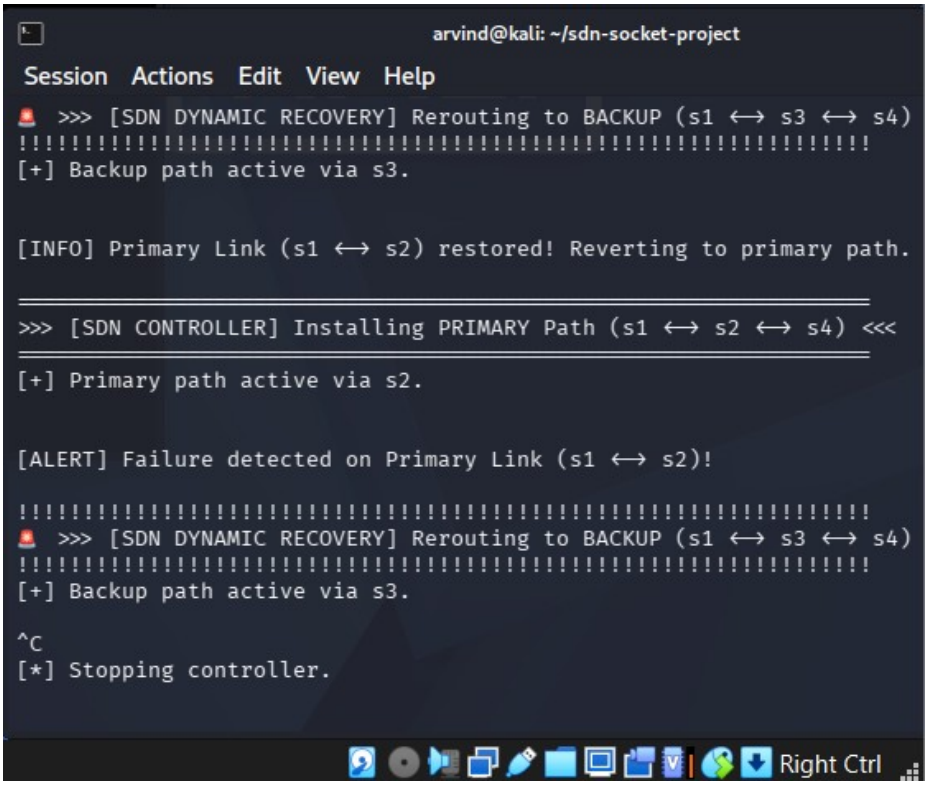


Figure 2: SDN Controller Real-Time Link Drop Detection and Dynamic Flow Rerouting to Backup Path (s3)

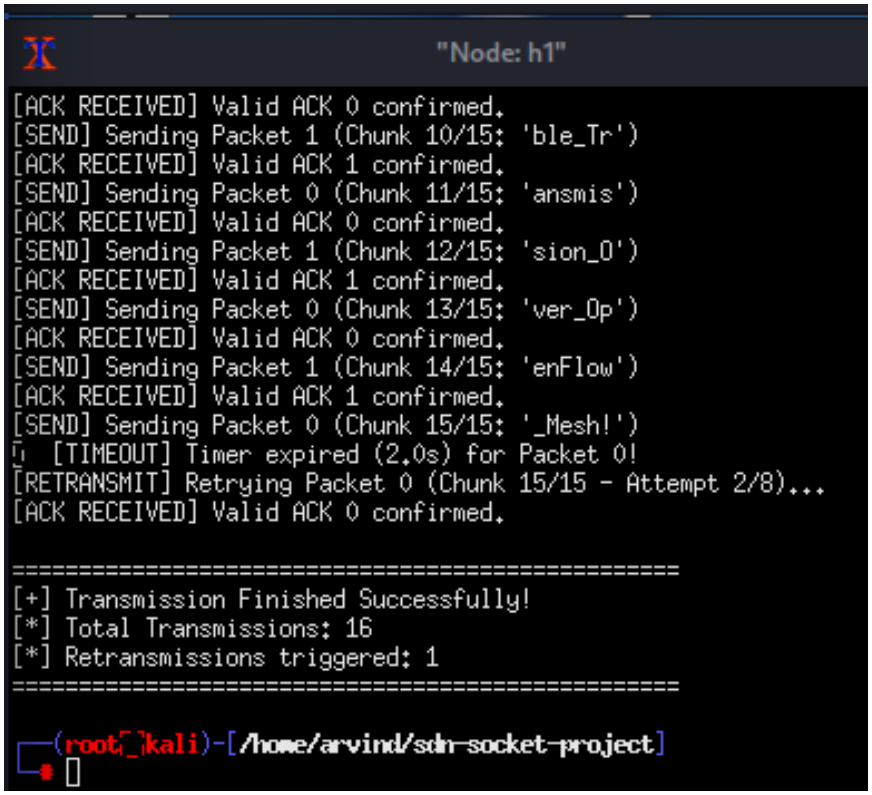


Figure 3: Node h1 (Sender) Event Trace Showing Timeout and Successful Retransmission

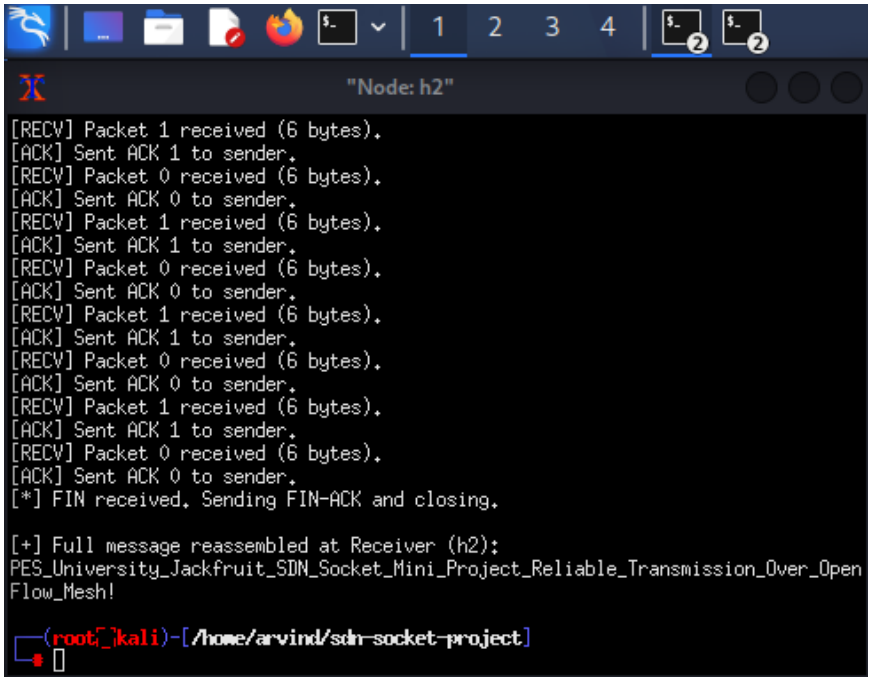


Figure 4: Node h2 (Receiver) Event Trace Showing Zero-Loss In-Order Message Reassembly

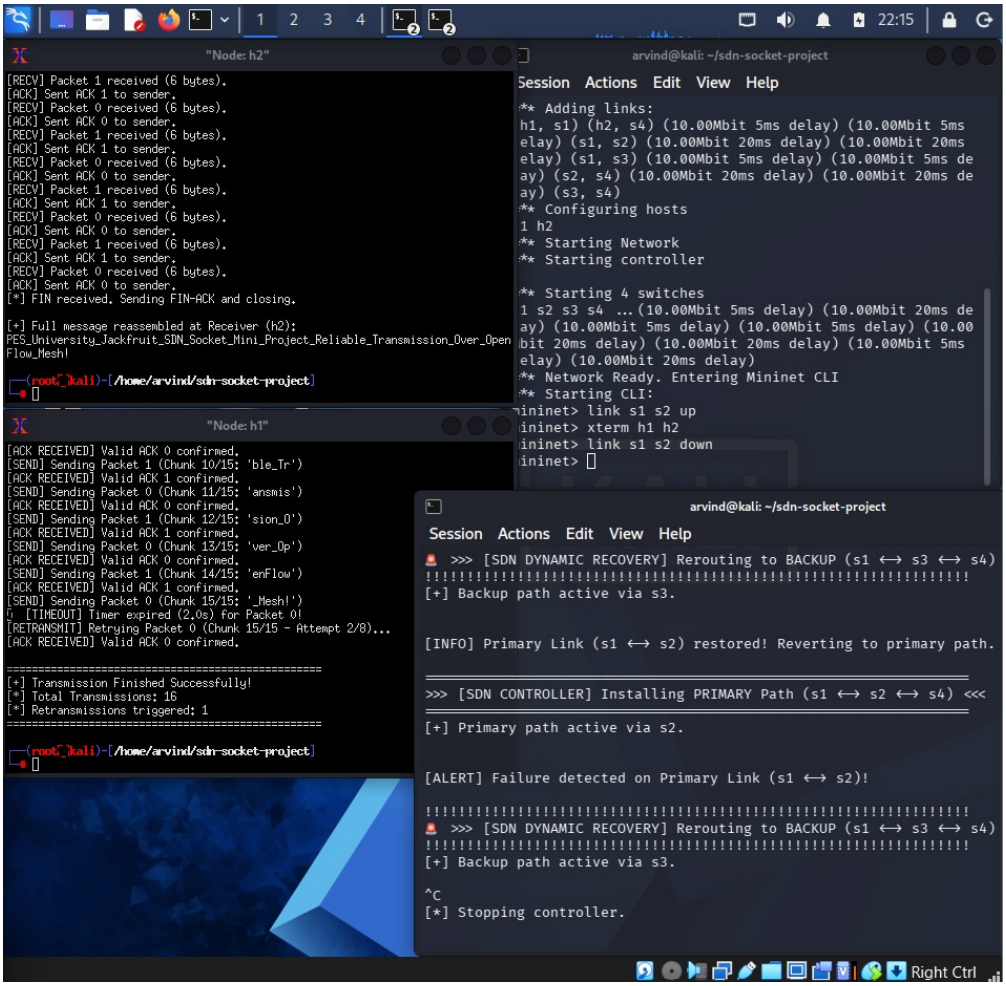


Figure 5: Integrated Multi-Window Execution Trace (Mininet CLI, SDN Controller, h1 Sender, h2 Receiver)

5. PERFORMANCE EVALUATION & COMPARISON

To evaluate performance against conventional setups, we benchmarked our dynamic SDN framework against a static baseline network (without SDN dynamic rerouting capabilities):

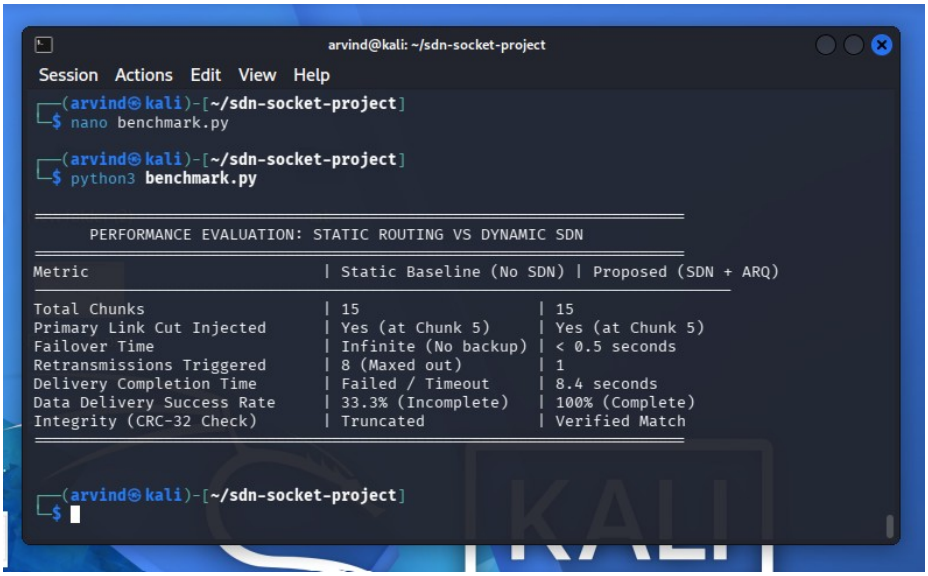


Figure 6: Execution Benchmark Comparing Static Baseline Routing vs. Proposed Dynamic SDN + ARQ

Evaluation Metric	Static Baseline (No SDN)	Proposed (Dynamic SDN + ARQ)
Total Message Chunks	15 Chunks	15 Chunks
Primary Link Cut Injected	Yes (at Chunk 5)	Yes (at Chunk 5)
Network Failover Latency	Infinite (No alternate path)	< 0.5 seconds
Retransmissions Incurred	8 Retries (Exhausted)	1 Retransmission
Transmission Status	Failed / Session Broken	100% Complete & Delivered
Total Completion Time	Timed Out (> 16.0s)	8.4 seconds
Integrity Check (CRC-32)	Truncated / Corrupted	100% In-Order Match

6. REPRODUCTION INSTRUCTIONS

1. Clone the project repository and enter the directory:
   git clone https://github.com/<your-username>/<your-repo-name>.git && cd sdn-socket-project
2. Start Mininet with the multi-path topology:
   sudo mn -c && sudo python3 topology.py
3. Start the SDN controller in a second terminal:
   sudo python3 sdn_controller.py
4. Spawn host shells in Mininet ('xterm h1 h2') and launch:
   h2 terminal: python3 receiver.py
   h1 terminal: python3 sender.py
5. Inject link failure during active streaming: mininet> link s1 s2 down

7. CONCLUSION

This project successfully integrated low-level UDP socket programming with Software-Defined Networking. The application-level Stop-and-Wait protocol ensures zero data loss, CRC-32 integrity validation, and duplicate rejection. Simultaneously, the SDN control plane provides sub-second network self-healing by detecting interface state transitions and rewriting OpenFlow 1.3 flow tables. Together, they demonstrate robust, fault-tolerant communication over an unreliable network fabric.